

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment**

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	RASHMI NAURENE	Reg. No.:	192521357
Date:	9.08.2026	Duration:	60 minutes

Code of Conduct

I _____ RASHMI NAURENE(192521357) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Rashmi _____

Annexure A – Git & Docker Technical Assignment**Instructions to Students:**

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions**1. Problem Analysis**

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.
- 5. **Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.
- 6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

1 · Problem Analysis

The assigned problem is the Employee Skill-Gap Analysis and Training Recommendation System. Organizations need a way to identify employee skill gaps by comparing recorded employee competencies against job-role requirements, and to recommend suitable training as a result. The system is intended to collect employee information such as performance data, certifications, assessments, and project experience; identify competency deficiencies; recommend training; track learning progress; and provide HR and manager analytics.

1.1 Scope of This Assignment

This assignment does not require building the application from scratch. The student forked an existing public GitHub repository that already implements an employee-management application in this problem domain, and then performed the Git version-control and Docker containerization work required by the assignment brief on top of that forked codebase. The sections below distinguish explicitly between what already existed in the forked repository and what the student added or changed.

1.2 Objectives Addressed by the Assignment Work

- Demonstrate a working Git branching and collaboration workflow (fork, branch, commit, push, pull request, merge, conflict resolution) on a real codebase.
- Introduce a small, well-scoped code contribution (a skill-gap data model) representative of the assignment's problem domain.
- Containerize the existing application and its PostgreSQL dependency using Docker and Docker Compose, and demonstrate that the containerized stack runs successfully.

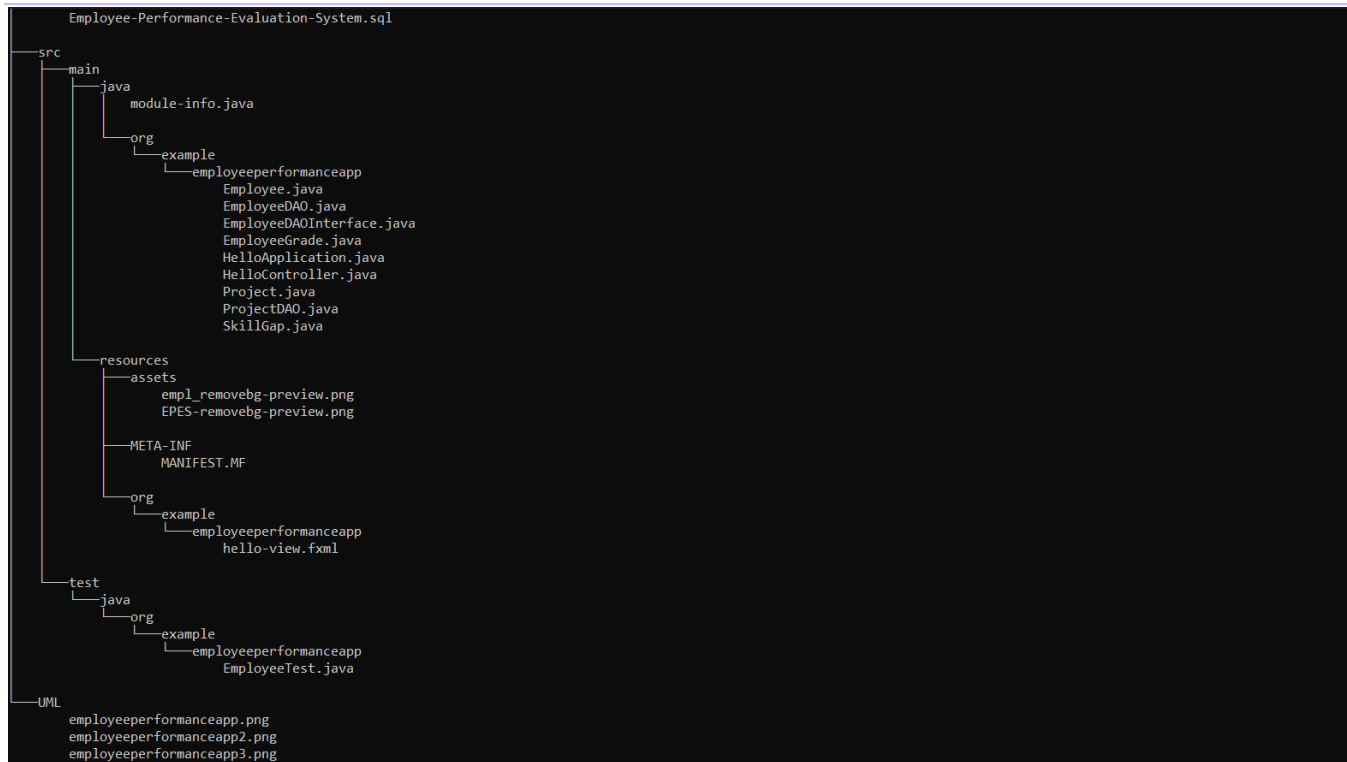
2 · Project Structure Design

2.1 Pre-Existing Structure (Forked Repository)

The forked repository is a Java/Maven JavaFX application. Its pom.xml declares a Maven project targeting Java 23, using JavaFX 21.0.3, the PostgreSQL JDBC driver 42.7.4, JUnit 5, and the javafx-maven-plugin. The Java source files present in the repository are:

```
src/main/java/.../Employee.java
src/main/java/.../EmployeeDAO.java
src/main/java/.../EmployeeDAOInterface.java
src/main/java/.../EmployeeGrade.java
src/main/java/.../HelloApplication.java
src/main/java/.../HelloController.java
src/main/java/.../Project.java
src/main/java/.../ProjectDAO.java
src/main/java/.../module-info.java
src/test/.../EmployeeTest.java
src/main/resources/ (JavaFX FXML/resource files)
pom.xml
```

The exact sub-package path for each file (e.g., the full path under src/main/java/) was not confirmed in the information provided for this report; the package path org.example.employeeperformanceapp is confirmed only for the



2.2 Purpose of Pre-Existing Files (Inferred from Naming)

File	Likely Purpose (by naming convention)
Employee.java	Domain/entity class representing an employee record.
EmployeeDAO.java	Data Access Object implementation handling persistence operations for Employee records, likely against the PostgreSQL database via the declared JDBC driver.
EmployeeDAOInterface.java	Interface defining the DAO contract that EmployeeDAO.java implements, following the standard DAO design pattern.
EmployeeGrade.java	Domain class or enumeration related to grading/classification of an employee.
HelloApplication.java	The JavaFX Application entry point (extends javafx.application.Application), responsible for launching the GUI.
HelloController.java	JavaFX FXML controller class handling UI events for the main view.
Project.java / ProjectDAO.java	Domain class and corresponding DAO for project-related records, following the same pattern as Employee/EmployeeDAO.
module-info.java	Java Platform Module System descriptor, required by the javafx-maven-plugin/module setup declared in pom.xml.
EmployeeTest.java	JUnit 5 test class for the Employee-related logic.
src/main/resources/	Holds JavaFX resource files (typically FXML layout files and/or CSS/images referenced by the controllers).

2.3 Addition Made by the Student

The student added one new Java class as part of the assignment's skill-analysis feature:

```
src/main/java/org/example/employeeperformanceapp/SkillGap.java
```

This class models an employee's current skill level against the level required for their role. Its stated fields and method are:

- `employeeId` — identifies which employee the record belongs to.
- `skillName` — the name of the competency being measured.
- `requiredLevel` — the proficiency level required for the employee's role.
- `currentLevel` — the employee's recorded proficiency level.
- `getGap()` — a method returning the difference between `requiredLevel` and `currentLevel`, i.e., the computed skill gap.

This directly supports the assignment's problem domain: it is the data model needed to represent "identify deficiencies" from Section 1, expressed as a concrete, gradeable Git contribution rather than a full application feature.

3 · Git Repository Initialization

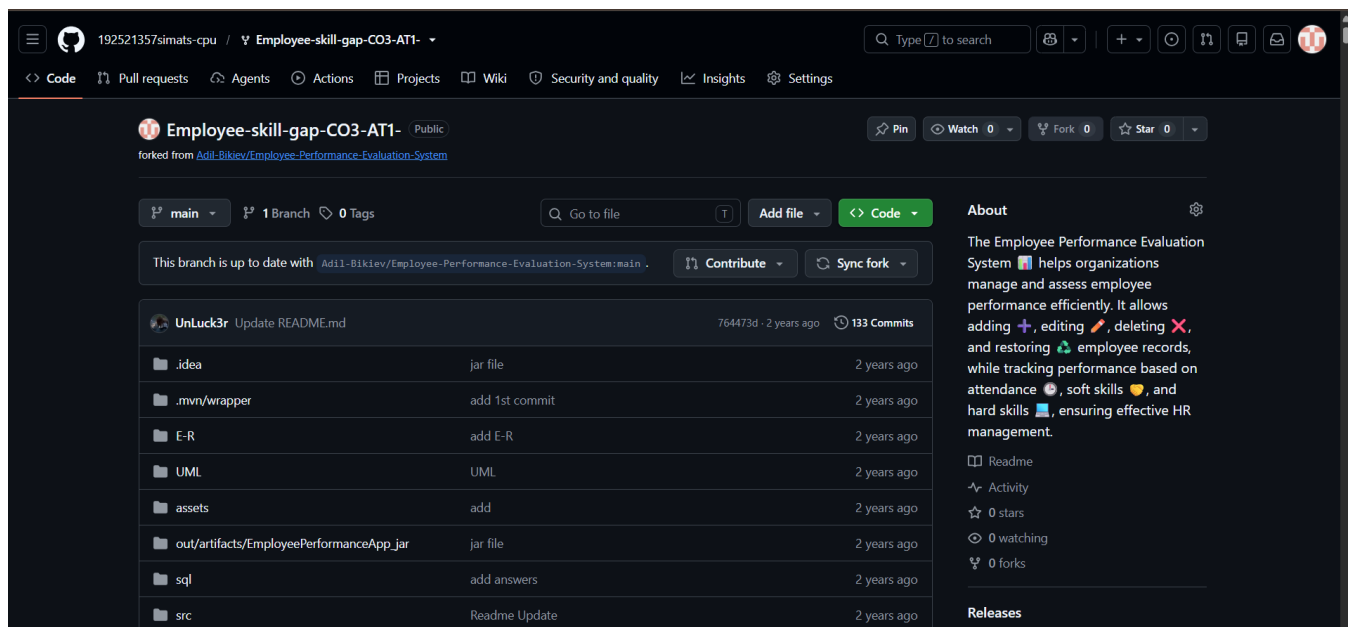
Because the codebase is a fork of an existing public repository, initialization for this assignment means forking and cloning, not running `git init` on an empty directory. The repository was forked to:

```
https://github.com/192521357simats-cpu/Employee-skill-gap-CO3-AT1-.git
```

and cloned locally with:

```
$ git clone https://github.com/192521357simats-cpu/Employee-skill-gap-CO3-AT1-.git
```

The clone completed successfully, giving the student a local working copy that includes the full commit history of the original repository up to the point of forking, plus a connection (origin) to their own fork on GitHub for pushing subsequent work.



3.1 Purpose of Essential Files Present in the Repository

File / Directory	Purpose
.git/	Created automatically by the clone operation; stores the full object database and history inherited from the forked repository.
README.md	Present in the repository — confirmed because it was the file used to create and resolve a controlled merge conflict (Section 5). Its general purpose is to document the project for anyone opening the repository.
pom.xml	The Maven project descriptor; declares the Java version, JavaFX version, PostgreSQL driver, JUnit, and the javafx-maven-plugin used to build and run the application.
.gitignore	[Screenshot/evidence required] — its presence and exact contents in this forked repository were not confirmed in the information provided.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\RASHMI NAURENE> git remote -v
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\RASHMI NAURENE> cd "C:\Windows\system32\Employee-skill-gap-C03-AT1-"
PS C:\Windows\system32\Employee-skill-gap-C03-AT1-> git remote -v
origin  https://github.com/192521357simats-cpu/Employee-skill-gap-C03-AT1-.git (fetch)
origin  https://github.com/192521357simats-cpu/Employee-skill-gap-C03-AT1-.git (push)
PS C:\Windows\system32\Employee-skill-gap-C03-AT1->
```

4 · Git Branching Strategy

Starting from the forked repository's existing main branch, the student created three additional branches:

- develop — integration branch for merging finished feature work.
- feature/skill-analysis — isolated branch for the SkillGap.java addition.
- feature/docker — isolated branch for the Dockerfile, .dockerignore, and docker-compose.yml additions.

The intended flow is main → develop → feature branches, with feature branches merged back into develop once complete.

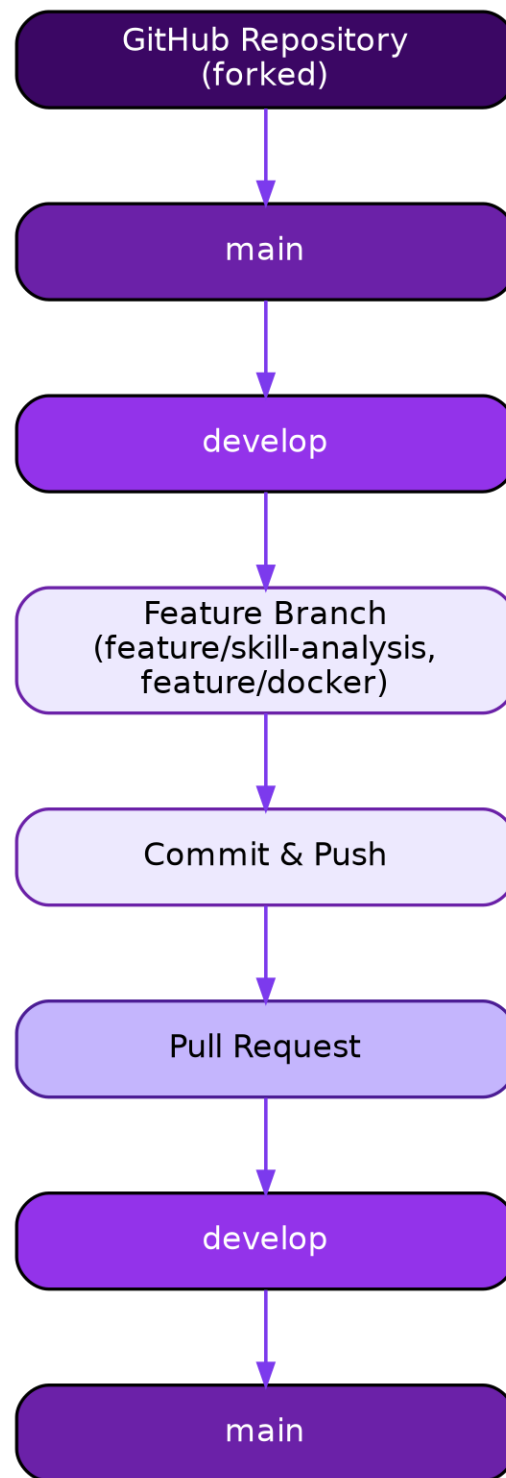
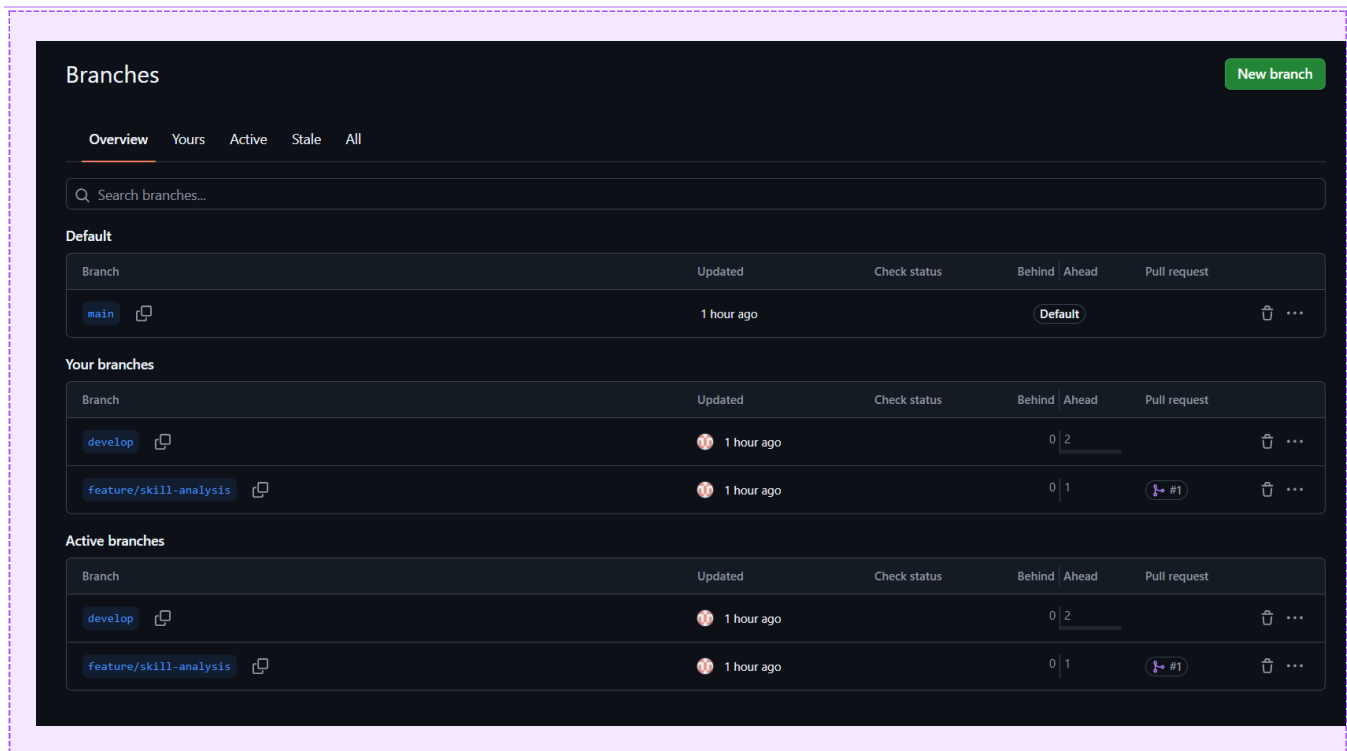


Figure 1 — Git branching and Pull Request workflow used for this assignment.



4.1 Justification for This Branching Model

- main is kept as the stable reference point, consistent with the fact that it already existed as the default branch of the forked repository before this assignment's work began.
- develop provides a single integration point for the two independent pieces of assignment work (the skill-analysis model and the Docker setup), so each can be reviewed and merged without directly touching main.
- Separate feature branches (feature/skill-analysis, feature/docker) isolate two unrelated concerns — a Java code addition and a containerization setup — so that a problem in one does not block the other.

5 · Version Control Workflow

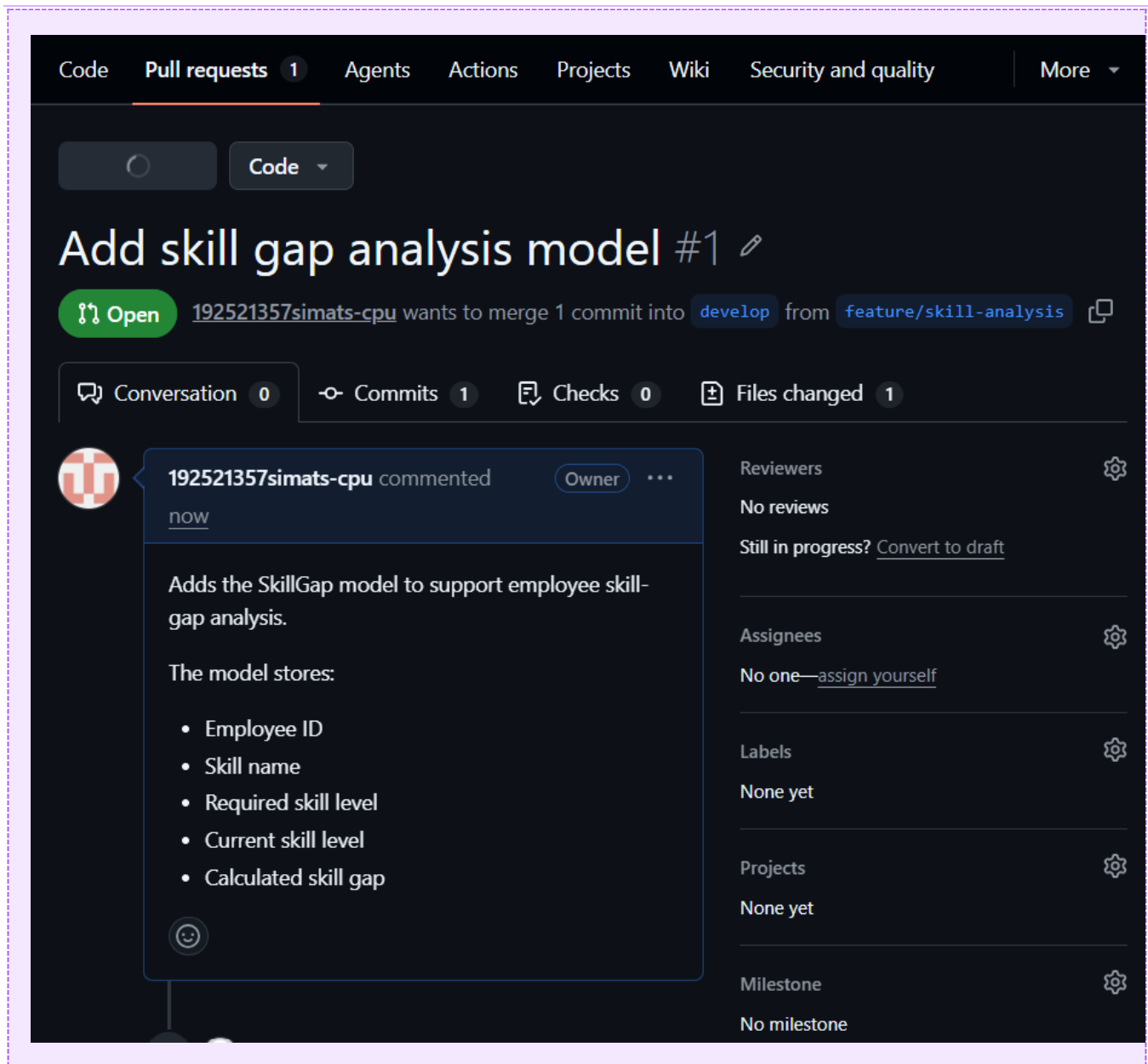
The following Git operations were demonstrated as part of this assignment: branch creation, feature branch development, commits, pushing branches to the remote, opening a Pull Request, merging a feature branch into develop, creating and resolving a merge conflict, synchronizing with the remote repository, and reviewing history via `git log / graph`.

5.1 Skill-Analysis Feature

On feature/skill-analysis, the student added SkillGap.java and committed it with the message:

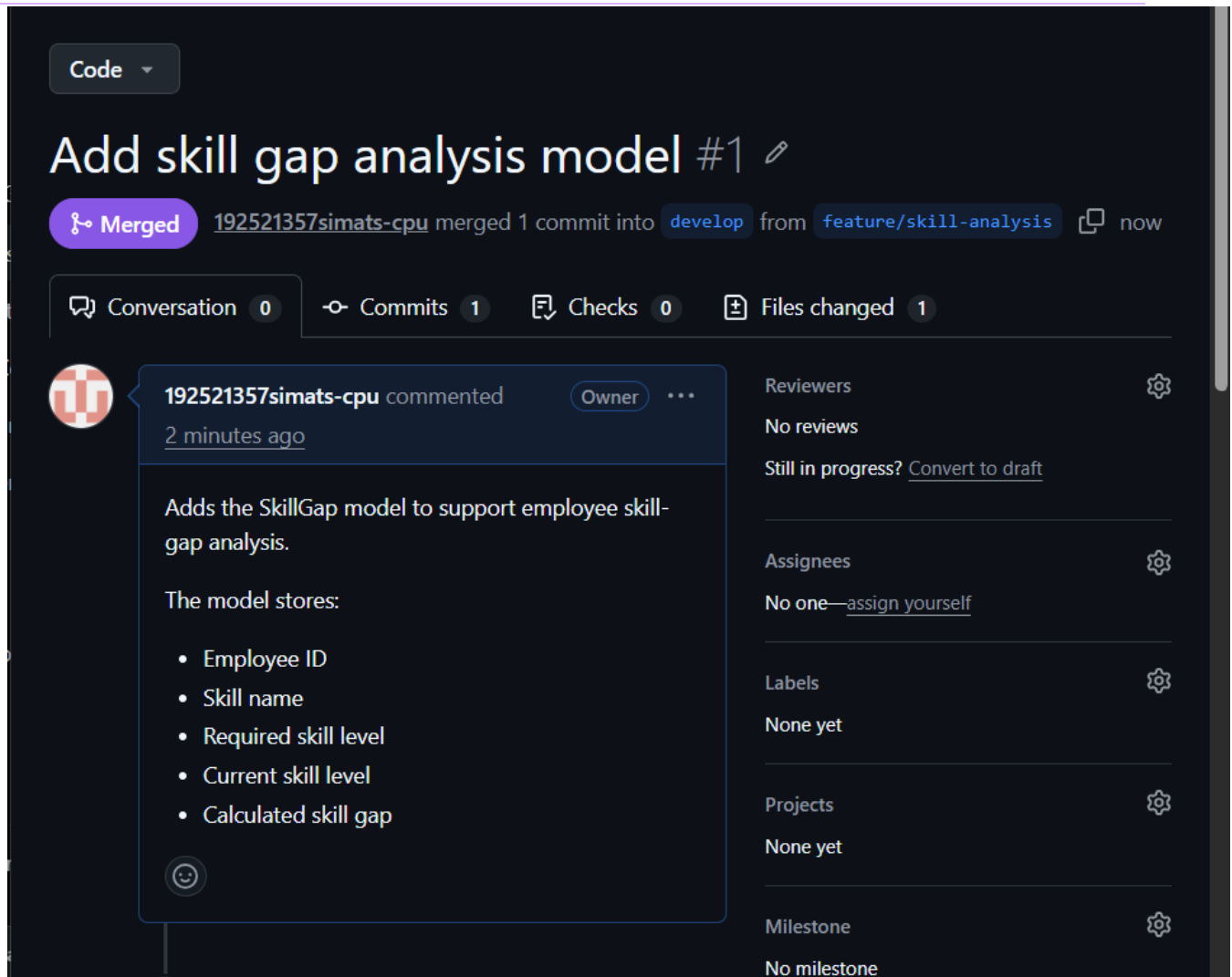
```
$ git add src/main/java/org/example/employeeperformanceapp/SkillGap.java
$ git commit -m "Add skill gap analysis model"
$ git push -u origin feature/skill-analysis
```

This branch was then pushed to GitHub and merged into develop through a Pull Request.



5.2 Controlled Merge Conflict

A merge conflict was deliberately created and resolved using README.md. The exact conflicting content and the specific resolution were not included in the information provided for this report and should be taken from the corresponding screenshots rather than assumed.



5.3 Docker Feature Branch

The Dockerfile, .dockerignore, and docker-compose.yml (Sections 8–9) were developed on feature/docker and are treated in this report as part of that branch's history. The exact commit sequence and messages for this branch were not included in the information provided and should be confirmed from the actual git log.

5.4 Synchronization and History Review

Standard synchronization commands (git pull / git push / git fetch) were used to keep the local repository, the fork, and the relevant branches up to date during this workflow. The authoritative record of exactly which commands were run, in what order, is the student's terminal history and the repository's actual commit log.

```

* 2b3314b (HEAD -> develop, origin/develop) Merge pull request #1 from 192521357simats-cpu/feature/skill-analysis
|
| * 044cb46 (origin/feature/skill-analysis, feature/skill-analysis) Add skill gap analysis model
|/
* 764473d (origin/main, origin/HEAD, main) Update README.md
* 7a85dc6 Update README.md
* 2cca87e add answers
* 556422b Readme update
* dc377cc Readme E-R
* 1af31f2 Readme ER
* dff5109 Readme Update
|
| * b05c958 add sql
| * 6df07db add
| * d198b40 Update README.md
| * bfd98d4 add Validation
| * 88e7fb0 Readme
| * 976998c Readme
|/
* 0a6829a Merge branch 'main' of https://github.com/EMMMABK/Employee-Performance-Evaluation-System
|
| * 1a8c344 Update README.md
| * e4c8051 UML
|/
* 0f505e0 Update README.md
* 783d813 Update README.md
* 4abafbe add E-R
* 01062de Update Employee-Performance-Evaluation-System.sql
* 51c79ab add one entity
* 87f5990 add new entity
* d3d42dc new Table Insert
* 6fad8f3 add 2-nd entity
* 42c00c8 add modified SQL
* 787cad9 add Alert same ID exception
* 949e6e9 Update README.md
* aace859 Update README.md
* 3348f44 Update README.md
* 8ba1f93 jar file
* c5e4555 UNIT_Test_Readme
* 0929288 UNIT_Test
* c5e6f9a add remove comments
* 1128d57 Update README.md
* fbd3d65 Update README.md
* 5013755 readme
* 61fb28e UML
* 67b489c add jar file
* d157261 add SQL updating
* f9fb050 Update README.md

```

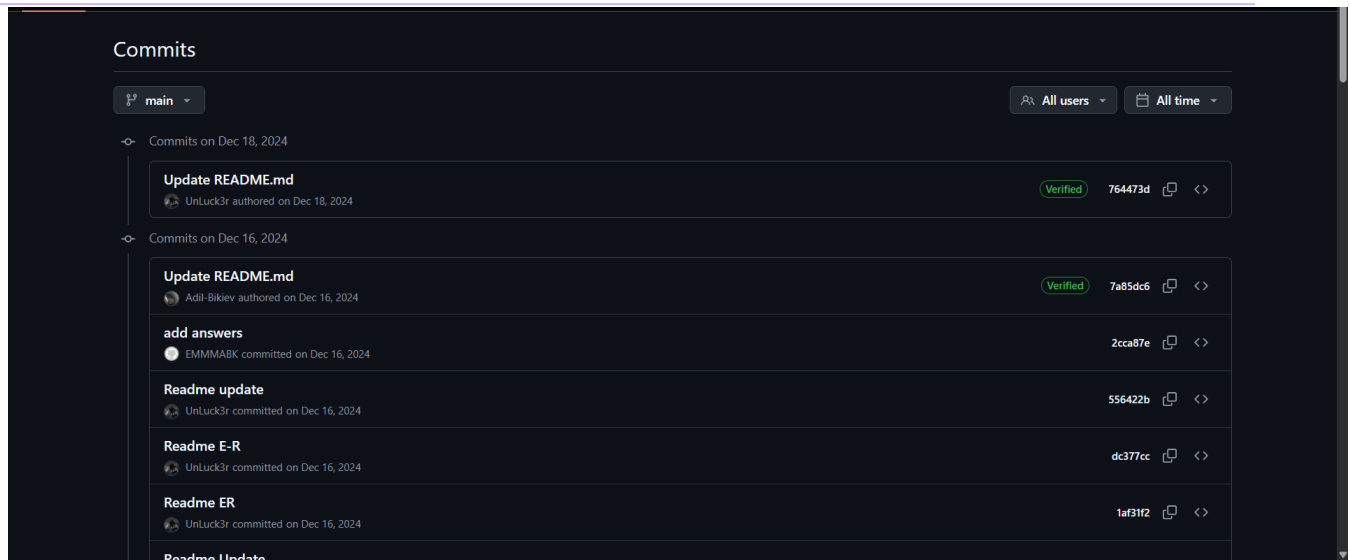
6 · Commit History Analysis

Only one commit message was provided as confirmed fact for this report:

```
Add skill gap analysis model
```

This message is short, written in the imperative mood ("Add..."), and states specifically what was introduced — consistent with standard Git commit-message practice, which favours a concise summary of what changed over a vague description like "update" or "fix stuff". This directly aids project maintenance: a future contributor running git log can tell from this single line, without opening the diff, that the commit introduces the skill-gap model.

The remainder of the commit history (additional commits on feature/skill-analysis, feature/docker, the conflict-resolution commit, and any commits on develop/main from merging) was not provided as text and must be taken from the actual repository. This report intentionally does not invent additional commit messages, hashes, or dates.



7 · Docker Environment Design

7.1 Why Docker Is Suitable for This Project

The forked application depends on a specific Java toolchain (Java 23, JavaFX 21.0.3, Maven) and a PostgreSQL database. Without containers, a grader or collaborator would need to install this exact combination locally before the application could even build. Docker packages the build environment and the database dependency into reproducible images, so the application can be built and its database dependency started with a small, fixed set of commands regardless of what is installed on the host machine.

7.2 Components Identified for Containerization

Component	Why It Was Containerized
Application (employee-skill-gap-app)	Bundles the Java 23 JDK, Maven, and the application source so the exact build/run environment declared in pom.xml is reproducible.
PostgreSQL (employee-postgres)	Uses the official postgres:16 image instead of requiring a manually installed local PostgreSQL server, and stores data in a named volume so it persists across container restarts.

8 · Dockerfile Development

The following Dockerfile was created on feature/docker to containerize the application build and run process, exactly as provided:

```
FROM eclipse-temurin:23-jdk

WORKDIR /app

COPY pom.xml .
```

```

COPY src ./src

RUN apt-get update && \
  apt-get install -y maven && \
  rm -rf /var/lib/apt/lists/*

RUN mvn clean package -DskipTests

CMD ["mvn", "javafx:run"]

```

Instruction	Purpose
FROM eclipse-temurin:23-jdk	Starts from an official Eclipse Temurin image providing a Java 23 JDK, matching the Java version declared in pom.xml.
WORKDIR /app	Sets /app as the working directory inside the container for all following instructions.
COPY pom.xml .	Copies the Maven project descriptor into the image first.
COPY src ./src	Copies the application source tree into the image.
RUN apt-get update && apt-get install -y maven && rm -rf /var/lib/apt/lists/*	Installs Maven inside the image (since the base image provides only the JDK, not Maven), then removes the apt package cache to keep the image smaller.
RUN mvn clean package -DskipTests	Builds the application inside the image at build time, skipping tests so the image build is not blocked by test execution.
CMD ["mvn", "javafx:run"]	The default command used to launch the JavaFX application when the container starts, via the javafx-maven-plugin declared in pom.xml.

The image was built successfully using:

```
$ docker build -t employee-skill-gap .
```

```

PS C:\Windows\system32\Employee-skill-gap-C03-AT1-> docker build -t employee-skill-gap .
[+] Building 2.4s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 266B
=> [internal] load metadata for docker.io/library/eclipse-temurin:23-jdk
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 65B
=> [1/6] FROM docker.io/library/eclipse-temurin:23-jdk@sha256:83177670e43c17769a5402a60601545318e45e63adf634fb5022340e516cdd4
=> => resolve docker.io/library/eclipse-temurin:23-jdk@sha256:83177670e43c17769a5402a60601545318e45e63adf634fb5022340e516cdd4
=> [internal] load build context
=> => transferring context: 1.97kB
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY pom.xml .
=> CACHED [4/6] COPY src ./src
=> CACHED [5/6] RUN apt-get update && apt-get install -y maven && rm -rf /var/lib/apt/lists/*
=> CACHED [6/6] RUN mvn clean package -DskipTests
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:f5448a5c017c2a7640d0889cee063237d938d6db9b9351c887a0e06dbf8d3f15
=> => exporting config sha256:329619a7d2572bdb2ec6811380471ff793d136c79447fc631ec72abe0ccf04fa
=> => exporting attestation manifest sha256:e70fcda21747691f759266dfa1240147247d5bd0f7be475a4185783160fde297
=> => exporting manifest list sha256:a1ad7c24c96f7afc4dda080c28ac1cb475fcaa643ea7562e9bb28f9ad2cf2d5d
=> => naming to docker.io/library/employee-skill-gap:latest
=> => unpacking to docker.io/library/employee-skill-gap:latest

View build details: docker-desktop:///dashboard/build/desktop-linux/desktop-linux/eqjoj6u0x9ywuuvoh414hypko

```

8.1 .dockerignore

The following .dockerignore file was also added, exactly as provided:

```
target
.git
.idea
*.iml
```

This prevents the Maven build output (target), Git metadata, and IDE-specific files from being sent to the Docker build context or copied into the image, keeping the image smaller and the build context transfer faster.

9 · Docker Compose Design

The following docker-compose.yml was created on feature/docker, exactly as provided:

```
services:
  postgres:
    image: postgres:16
    container_name: employee-postgres
    environment:
      POSTGRES_DB: employee_db
      POSTGRES_USER: employee_user
      POSTGRES_PASSWORD: employee_pass
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  app:
    build: .
    container_name: employee-skill-gap-app
    depends_on:
      - postgres
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: employee_db
      DB_USER: employee_user
      DB_PASSWORD: employee_pass

volumes:
  postgres_data:
```

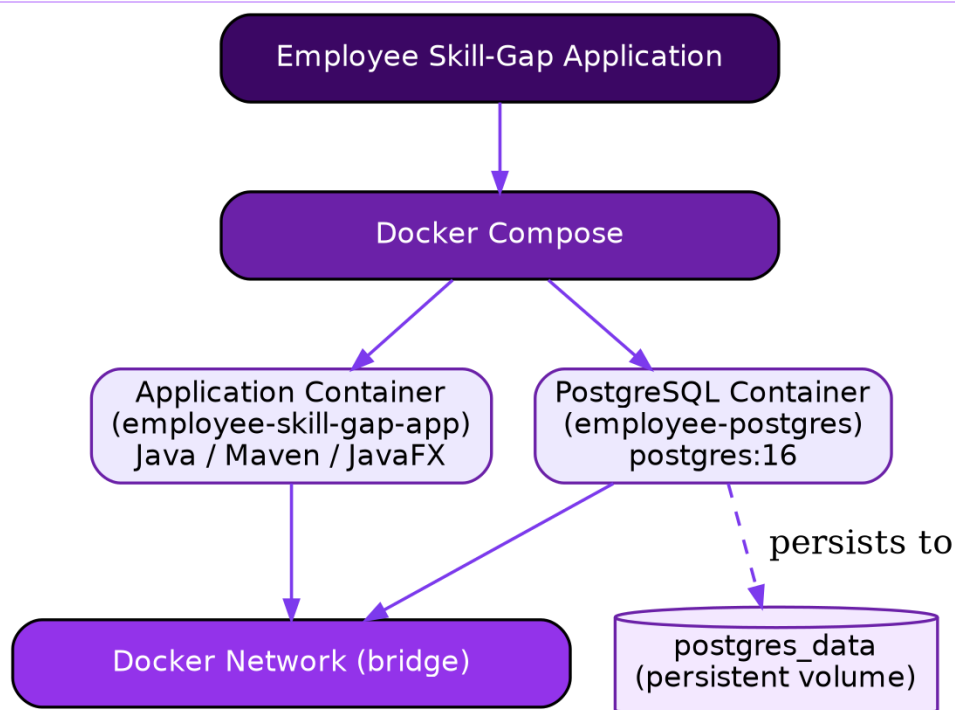


Figure 2 — Docker Compose architecture: application and PostgreSQL containers on a shared Docker network, with a persistent volume for database data.

9.1 Services, Networking, Volumes, and Dependencies

- **Services:** two services are defined — postgres (using the official postgres:16 image) and app (built from the local Dockerfile in Section 8).
- **Networking:** both services are placed on Compose's default network for this project, allowing app to reach the database at host postgres on port 5432, as reflected in the DB_HOST/DB_PORT environment variables.
- **Volumes:** a named volume, postgres_data, is mounted at PostgreSQL's data directory so database contents persist across container restarts.
- **Environment variables:** POSTGRES_DB/POSTGRES_USER/POSTGRES_PASSWORD configure the database on the postgres service; the matching DB_HOST/DB_PORT/DB_NAME/DB_USER/DB_PASSWORD variables are passed to app so it can connect to that same database.
- **Dependencies:** depends_on ensures postgres is started before app, matching the fact that the application requires the database to be reachable.

10 · Container Deployment and Testing

The image was built via Compose, and the stack was brought up in detached mode:

```
$ docker compose build
# application image built successfully

$ docker compose up -d
# Error: container name "/employee-skill-gap-app" is already in use
# (a container with this name already existed from an earlier `docker run`)
```

This container-name conflict was resolved by removing the pre-existing container and re-running Compose:

```
$ docker rm employee-skill-gap-app
$ docker compose up -d
# both containers started successfully
```

[Insert Screenshot: The container-name-conflict error, and the successful `docker compose up -d` after removing the old container]

10.1 Verifying the Running Containers

docker compose ps reported both containers running:

Container	Image	Status	Ports
employee-postgres	postgres:16	Up	0.0.0.0:5432->5432/tcp
employee-skill-gap-app	employee-skill-gap-co3-at1--app	Up	<i>[Screenshot/evidence required]</i>

The screenshot shows the Docker Desktop interface. At the top, it says 'Containers' with a 'Give feedback' link. Below this, there are two sections: 'Container CPU usage' and 'Container memory usage', both showing 'No containers are running.' and a 'Show charts' link. A search bar and a toggle for 'Only show running containers' are present. The main table lists running containers:

Name	Container ID	Image	Port(s)	CPU (%)	Last start	Actions
flask-container	09434f568c2b	flask-api	5000:5000	N/A	1 day ago	[Stop] [Refresh] [Restart] [Delete]
mywebsite	7b9f81e4a171	static-web	8080:80	N/A	1 day ago	[Stop] [Refresh] [Restart] [Delete]
nervous_kare	77476bdb2353	python		N/A	1 day ago	[Stop] [Refresh] [Restart] [Delete]
pensive_albatta	e10832eb52bc	ubuntu		N/A	15 days ago	[Stop] [Refresh] [Restart] [Delete]
employee-skill-gap-app	edf37906c5f0	employee-skill-gap-co3-at1--app		N/A	17 seconds ago	[Stop] [Refresh] [Restart] [Delete]

On the right side, there is a vertical list of container names and their CPU usage, including 'ktop-linux' and 'employee-skill-gap-app'.

10.2 Database Connectivity Test

The PostgreSQL container's readiness was tested directly:

```
$ docker exec employee-postgres pg_isready -U employee_user -d employee_db
employee_db:5432 - accepting connections
```


11 · Benefits of Git and Docker for This Project

Benefit	How It Applied Here
Collaboration	Working on separate feature/skill-analysis and feature/docker branches meant the Java model addition and the containerization work could proceed as independent, reviewable units against the forked codebase.
Version Management	The forked repository's full history was preserved by cloning rather than starting a fresh repository, and the SkillGap.java addition is tracked as its own commit rather than an untracked local change.
Portability	Docker removed the need to manually install Java 23, Maven, and PostgreSQL 16 on any machine used to build or grade this project — docker build and docker compose up reproduce the environment directly from the Dockerfile and docker-compose.yml.
Deployment	docker compose up -d starts both the application and its database dependency together, in the correct order, with a single command.
Maintainability	Configuration (database name, user, credentials, ports) is expressed as environment variables in docker-compose.yml rather than hard-coded, making it easier to change without editing source code.

12 · Challenges and Improvements

12.1 Challenges Actually Encountered

- Container name conflict: docker compose up -d initially failed because a container named employee-skill-gap-app already existed from an earlier docker run invocation. This was resolved by removing the old container (docker rm employee-skill-gap-app) before re-running Compose (Section 10).
- JavaFX GUI in a container: the application is a JavaFX desktop GUI rather than a headless service, which raises a real consideration for containerized deployment — a container does not have a display attached by default, so running a GUI application's window from inside it typically requires additional configuration (e.g., X11 forwarding) that was not confirmed as part of this submission's evidence. This is noted here as a genuine open consideration rather than a claimed solution.

12.2 Suggested Improvements and Future Enhancements

- Add an explicit healthcheck: on the postgres service in docker-compose.yml so app only starts once PostgreSQL is actually ready to accept connections, rather than relying on depends_on's start-order-only guarantee.
- Document the exact resolution steps for the README.md merge conflict in the repository itself, so the Git history is self-explanatory for future contributors.
- Evaluate whether the JavaFX GUI should be replaced or supplemented with a headless/service mode for containerized environments, or whether a remote-display solution is the intended deployment path.
- Introduce a .env file (excluded via .gitignore) for the PostgreSQL credentials currently hard-coded in docker-compose.yml, to avoid committing plaintext passwords.

13 · Conclusion

This assignment applied Git version-control practices and Docker containerization to a forked, pre-existing Java/Maven JavaFX application in the Employee Skill-Gap Analysis problem domain. The student did not author the original application; the contribution for this assignment consisted of a Gitflow-style branching workflow (main / develop / feature branches), one new domain class (SkillGap.java) demonstrating the assignment's problem domain, a demonstrated merge-conflict resolution, and a complete Docker and Docker Compose setup that successfully builds and runs the application alongside a PostgreSQL database on a shared Docker network with persistent storage. The container-name conflict encountered during deployment was identified and resolved, and the database connection was verified directly with `pg_isready`. Sections marked [Screenshot/evidence required] throughout this report should be completed once the corresponding screenshots are available, at which point they can also be used to verify every claim above against the actual repository and terminal output.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25